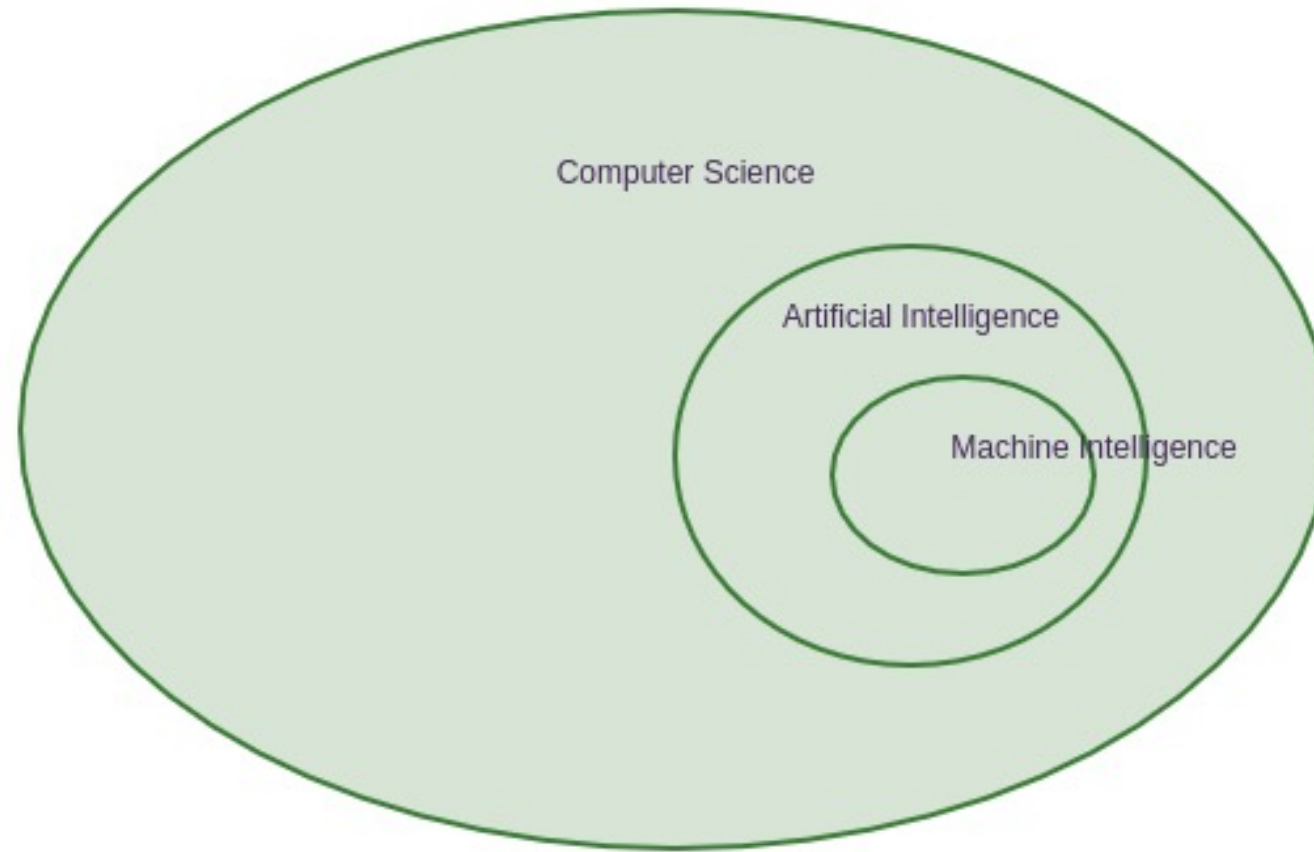


Modelos de Aprendizaje de Máquina (*Machine Learning*)

¿Qué es *Machine Learning (ML)*?



Proceso general

1 Entender el problema de negocio

¿Dónde hay una oportunidad de mejora?

2 Mapear el problema a uno de ML

No es necesario escoger el modelo final, sino hacer preguntas como ¿Es un problema de predicción? ¿Es regresión o clasificación? ¿Cómo podría verse la variable de interés?

3 Obtener e integrar los datos con los que se entrenará el sistema

Comúnmente, los datos provendrán de muchas fuentes y será necesario un esfuerzo grande para combinarlos.

4 Explorar y procesar los datos

La limpieza y el EDA son primordiales, pero también debe empezar a considerarse el proceso automatizado de preprocesamiento para un eventual despliegue.

5 Crear *features* o variables

Gran parte del éxito de un modelo de ML depende de haber usado o creado las variables (o *features*) correctas

6 Seleccionar los métodos de ML a probar

¿Cuáles modelos del tipo de problema de ML a resolver tendría sentido probar? Es importante considerar el tipo y cantidad de datos con que se cuenta, así como restricciones de implementación y tecnología.

7 Evaluar el desempeño de los modelos

Deben considerarse las métricas adecuadas: ¿todos los errores son igual de graves?

8 Desplegar, mantener y actualizar el sistema

Los modelos suelen requerir ser reentrenados regularmente, para poder aprender nuevos patrones. Asimismo, las implementaciones pueden requerir mantenimiento u optimizaciones.

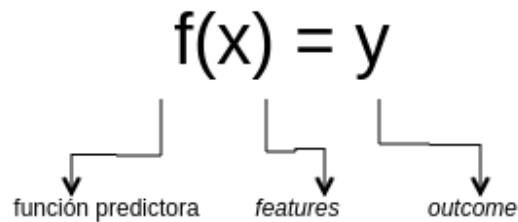
Tipos de aprendizaje en *ML*

	Descripción	Ejemplos
Supervisado	<i>Tenemos historia de donde aprender los errores y aciertos. Tenemos manera de supervisar la predicción.</i>	- ¿Esta transacción es fraude? Sí No - ¿Este paciente tendrá diabetes en los siguientes 2 años? Sí No - ¿Tendremos más de 1 millón de ventas el siguiente trimestre? Sí No - ¿Este ecocardiograma muestra una enfermedad cardiaca? Sí Revisión de cardiólogo No
No Supervisado	<i>Tenemos historia de donde obtener patrones.</i>	- ¿Qué tipos de clientes tenemos? - ¿Cuáles son las características de las audiencias que nos visitan? - ¿Cuáles son las características de los pacientes que vienen? - ¿Qué enfermedades suceden juntas?
Por Refuerzo	<i>Tenemos objetivos a los cuales queremos llegar y tenemos formas específicas de dar un refuerzo cuando lo hacemos bien -y cuánto refuerzo-, y un castigo cuando nos equivocamos -y cuánto castigo- optimizamos a alcanzar la meta.</i>	- Estrategias de atención al cliente - Estrategias de recuperación de cartera - Asignación de líneas de crédito

Aprendizaje Supervisado

¿Qué busca el aprendizaje supervisado?

Contamos con datos históricos "calificados" que nos permiten supervisar las predicciones que los modelos hacen de las observaciones.



El objetivo consiste en encontrar la f que minimice el error para recuperar la y .

$f(\text{🍏}) = \text{"apple"}$

$f(\text{🐕}) = \text{"dog"}$

Si la y es numérica, el problema es supervisado de regresión,
si la y es categórica, el problema es supervisado de clasificación.

¿Regresión o clasificación?

Regresión

Clasificación

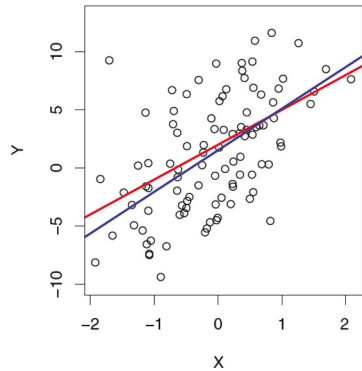
1. Determinar si un email es spam o no
2. Identificar si la persona tratando de desbloquear un iphone es la dueña
3. Estimar el tiempo que tardará un paquete en ser entregado
4. Predecir el precio de una casa
5. Predecir la inscripción de alumnos a MEK el próximo año
6. Predecir el nivel de riesgo de un embarazo
7. Identificar si un tumor es benigno o maligno
8. Predecir la demanda de televisiones en el Buen Fin
9. Estimar la probabilidad de que el elevador B del edificio se averíe en las próximas dos semanas
10. Determinar si imágenes satelitales de campos agrícolas muestran *irregularidades legales*

Regresión

Clasificación

Modelos de regresión

Regresión Lineal



Describe relaciones lineales entre dos (o más) variables. Es directamente interpretable como “un incremento en una unidad en la variable X, causa un incremento (o disminución) en la variable Y de n unidades.

Utiliza variables numéricas (X's, llamadas “independientes”) para predecir una variable de interés (Y, llamada “dependiente”), también numérica.

Proporciona coeficientes para cada variable independiente, los cuales son directamente interpretables.

Su principal obstáculo es que depende de relaciones lineales. Por ejemplo, si el impacto de una variable independiente tiene forma de U, es decir, aumenta el valor de la variable Y cuando toma valores muy bajos o muy altos, esto no será capturado en el modelo, e incluso podría empeorar el desempeño.

Código ejemplo

```
# Importar las bibliotecas necesarias
import numpy as np
from sklearn.linear_model import LinearRegression

# Crear datos de muestra
X = np.random.rand(100, 1) * 10 # Valores de entrada
y = 2.5 * X + 1.3 + np.random.randn(100, 1) * 2 # Valores de salida con una relación lineal y ruido

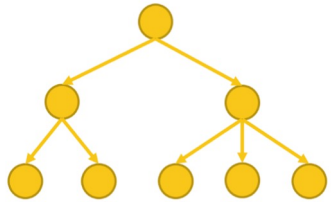
# Inicializar y entrenar el modelo
modelo_lineal = LinearRegression()
modelo_lineal.fit(X, y)

# Predecir valores
y_pred = modelo_lineal.predict(X)

# El modelo ahora puede usarse para predecir valores no vistos.
```

Modelos de regresión

Árbol de Regresión¹



Construye nodos basados en puntos de corte para ciertas variables, agrupando observaciones en nodos terminales con cierto valor numérico de Y asignado. Prioriza variables automáticamente, y puede no usar todas aquellas provistas en la fase de entrenamiento.

Utiliza variables, tanto numéricas como categóricas, para predecir una variable numérica. Agrupa tal variable en rangos, y crea nodos de decisión con cortes en variables para llegar a una predicción. Puede no utilizar todas las variables disponibles, y puede trabajar con datos faltantes.

Aunque no produce coeficientes, es altamente interpretable y puede ser implementado incluso sin una computadora, dado que son preguntas “sí/no” que llevan a una respuesta.

Dado que crea rangos para una variable numérica, su margen de error es mucho más grande que otros modelos de regresión.

Código ejemplo

```
# Importar las bibliotecas necesarias
import numpy as np
from sklearn.tree import DecisionTreeRegressor

# Crear datos de muestra
X = np.random.rand(100, 1) * 10 # Valores de entrada
y = np.sin(X) + np.random.randn(100, 1) * 0.1 # Valores de salida basados en una función no lineal y ruido

# Inicializar y entrenar el modelo
modelo_arbol = DecisionTreeRegressor()
modelo_arbol.fit(X, y)

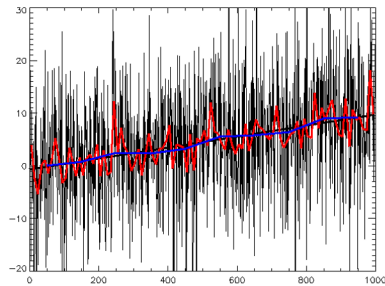
# Predecir valores
y_pred = modelo_arbol.predict(X)

# El árbol de decisión ahora puede usarse para predecir valores no vistos.
```

1. Los árboles de decisión que predicen una variable continua son llamados árboles de regresión, mientras que aquellos que predicen una categoría son llamados árboles de clasificación.

Modelos de regresión

Serie de Tiempo



Analiza puntos en el tiempo, con un orden natural temporal. En general, encuentra la autocorrelación con observaciones anteriores, y extrae la tendencia, ciclos y temporalidad de los datos, para predecir las observaciones futuras.

Utiliza el histórico de una variable numérica para predecir observaciones futuras de ella misma. El orden de los datos toma importancia. A medida que el modelo se vuelve mas complejo, crea componentes autorregresivos y de medias móviles.

Proporciona coeficientes, los cuales son directamente interpretables. También las tendencias,, ciclos y temporalidad pueden ser estudiadas.

Su principal obstáculo es que es sumamente sensible a cambios drásticos o picos. También, el modelo crece rápidamente en número total de variables al convertirse en multivariado.

Código ejemplo

En general, este tipo de modelos conllevan un proceso de creación de variables/features diferente, y considerablemente más complejo, que los modelos “estándar” de aprendizaje de máquina.

La librería común para modelos de machine learning, scikit-learn, no contiene módulos para series de tiempo.

En este curso no se ahondará en este tipo de modelos, dada su alta complejidad y lógica diferente. Sin embargo, son la herramienta adecuada en muchas ocasiones y es importante conocer su existencia.

Aquí hay un tutorial muy completo, de Kaggle, para aprender series de tiempo:

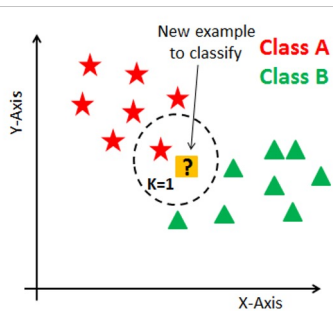
<https://www.kaggle.com/code/prashant111/complete-guide-on-time-series-analysis-in-python>

Regresión

Clasificación

Modelos de clasificación

K- Vecinos



Calcula una predicción tomando en cuenta k observaciones similares (“vecinos”), generalmente usa el promedio o la moda de sus valores de Y para el resultado.

No produce un modelo.

Utiliza variables numéricas para predecir una variable Y numérica, binaria o categórica. Para una nueva observación, calcula la distancia a todas las demás, obtiene los k vecinos más cercanos y asigna una función de sus valores de Y (promedio, moda, etc.) como predicción.

Es relativamente resiliente a errores o datos extremos. Mientras que no proporciona coeficientes, es un modelo con una premisa simple y fácil de explicar.

El principal obstáculo es la necesidad de calcular distancias a todos los puntos en los datos originales para cada nueva observación, así que es computacionalmente lento cuando los datos son grandes.

Código ejemplo

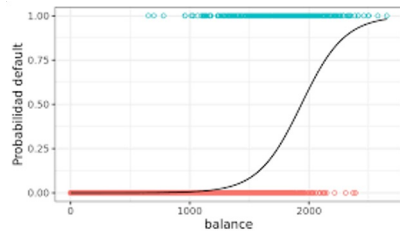
```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Crear conjunto de datos
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Modelo KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
predicciones_knn = knn.predict(X_test)
```

Modelos de clasificación

Regresión Logística



Clasifica observaciones binarias, generalmente pertenencia a un grupo. La predicción para cada observación es un valor entre (0,1) y se interpreta como la probabilidad de pertenencia. Proporciona coeficientes interpretables.

Utiliza variables numéricas (X's, llamadas "independientes") para predecir una variable de interés (Y, llamada "dependiente"), binaria. Generalmente se interpreta como pertenencia a una categoría, por ejemplo [fraude | no fraude] o [*churn* | no *churn*].

Proporciona coeficientes para cada variable independiente, los cuales son interpretables. La predicción, que se encuentra en el intervalo (0,1) se interpreta como la probabilidad de pertenencia al grupo.

Su principal obstáculo es que requiere de una elección de probabilidad mínima para predecir pertenencia. También, como la regresión lineal, no captura relaciones complejas.

Código ejemplo

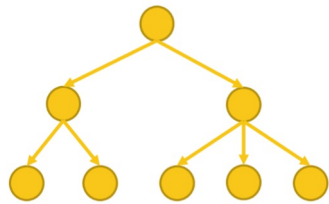
```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Crear conjunto de datos
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Modelo de Regresión Logística
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)
predicciones_log_reg = log_reg.predict(X_test)
```

Modelos de clasificación

Árbol de Clasificación¹



Construye nodos basados en puntos de corte para ciertas variables, agrupando observaciones en nodos terminales con una cierta categoría asignada.

Prioriza variables automáticamente, y puede no usar todas aquellas provistas en la fase de entrenamiento.

Utiliza variables, tanto numéricas como categóricas, para predecir una variable binaria o con categorías. Crea nodos de decisión con cortes en variables para llegar a una predicción. Puede no utilizar todas las variables disponibles, y puede trabajar con datos faltantes. Captura relaciones complejas entre variables.

Aunque no produce coeficientes, es altamente interpretable y puede ser implementado incluso sin una computadora.

Es muy sensible a ruido en los datos. Por otro lado, es un modelo con relativamente bajo desempeño comparado con *random forest*, el cual debe ser considerado si el caso de uso lo permite.

1. Los árboles de decisión que predicen una variable continua son llamados árboles de regresión, mientras que aquellos que predicen una categoría son llamados árboles de clasificación.

Código ejemplo

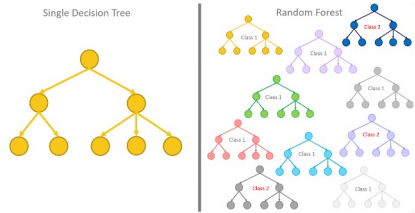
```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Crear conjunto de datos
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Modelo Árbol de Decisión
arbol_clasificacion = DecisionTreeClassifier()
arbol_clasificacion.fit(X_train, y_train)
predicciones_arbol = arbol_clasificacion.predict(X_test)
```

Modelos de clasificación

Random Forest



Los *Random Forests*, en español “Bosques Aleatorios” son un ensamble de varios árboles de decisión creados con subconjuntos aleatorios de los datos de entrenamiento, con una predicción para cada observación por voto de cada árbol.

Construye una alta cantidad de árboles de clasificación, cada uno con una muestra aleatoria de los datos. De esta manera, obtiene una predicción binaria/de categoría para cada observación. Después, asigna la predicción más común como predicción global del modelo.

Puede trabajar con datos faltantes y encuentra relaciones complejas entre variables. Requiere recursos de cómputo mucho más altos que los modelos tradicionales.

Pierde la alta interpretabilidad y la baja necesidad de recursos computacionales de un árbol, a cambio de una alta ganancia en poder de predicción.

Código ejemplo

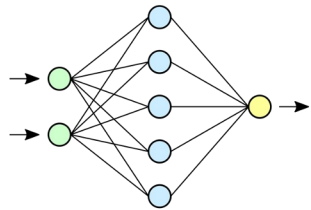
```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Crear conjunto de datos
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Modelo Random Forest
random_forest = RandomForestClassifier(n_estimators=100)
random_forest.fit(X_train, y_train)
predicciones_rf = random_forest.predict(X_test)
```


Modelos de clasificación

Red Neuronal¹



En general, este tipo de modelos tienen una capa de entrada (con un nodo por variable en los datos), la de procesamiento (con pesos y funciones que procesan o combinan las variables de entrada) y una de salida con la predicción por observación (con un nodo por variable a predecir).

Como su nombre lo indica, es una red comprendida de nodos conectados a otros nodos (como neuronas) organizados en capas. La capa de entrada contiene las variables a utilizar. La oculta, contiene funciones que se aplican a uno o varios nodos de entrada. La de salida contiene la predicción binaria o categórica para la variable objetivo.

Puede tener más de un nodo de salida. Encuentra relaciones complejas entre variables. Es poco sensible a ruido y a datos faltantes.

Sin embargo, tiene baja interpretabilidad y requiere optimización de parámetros para encontrar el mejor modelo.

Código ejemplo

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

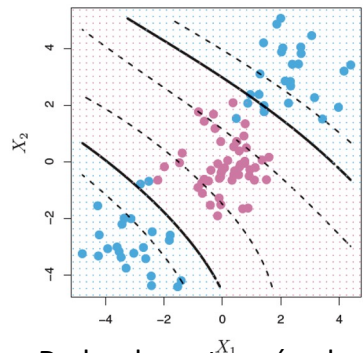
# Crear conjunto de datos
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Modelo Red Neuronal
red_neuronal = MLPClassifier(hidden_layer_sizes=(100, ), max_iter=1000)
red_neuronal.fit(X_train, y_train)
predicciones_nn = red_neuronal.predict(X_test)
```

1. Una manera sencilla de entender aprendizaje profundo, es como una red neuronal con más de una capa intermedia. Cada una de estas capas se vuelve más sofisticada que las anteriores, y puede encontrar patrones más complejos en los datos. Son modelos con muy baja interpretabilidad y necesidad de una cantidad muy elevada de datos para el entrenamiento.

Modelos de clasificación

Máquinas de Soporte Vectorial



Clasifica observaciones en dos grupos, creando una frontera de decisión basada en un vector. Esto permite fronteras de decisión complejas más allá de lineales (por ejemplo, un círculo dentro de otro círculo).

Dadas dos categorías, busca el vector que mejor las separa. Tiene gran flexibilidad respecto a qué puede ser este vector: una línea, un plano, un círculo, etc., así que puede separar categorías complejas.

Comparado con algoritmos más complejos, como redes neuronales, tienen un mejor desempeño cuando el volumen de datos es limitado.

Su principal obstáculo es que el vector depende de un parámetro llamado *kernel*; este debe ser elegido según el desempeño del modelo y el expertise de los roles técnicos. Además, el peso de cada variable no es constante para cada observación, así que la interpretación es difícil.

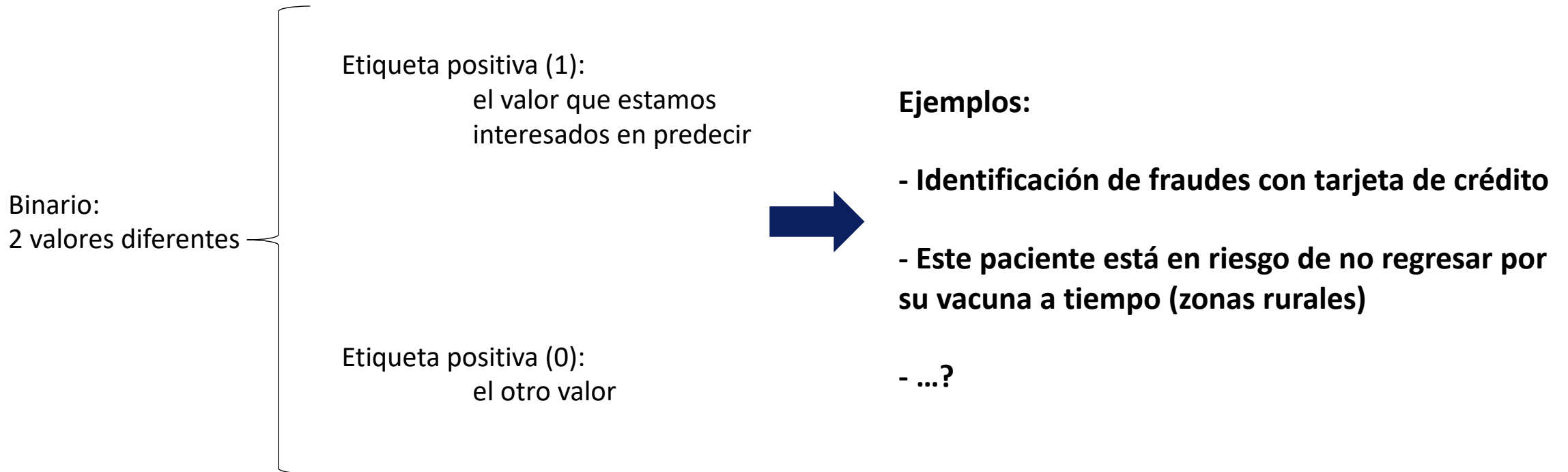
Código ejemplo

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

# Crear conjunto de datos
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Modelo SVM
svm = SVC(kernel='linear')
svm.fit(X_train, y_train)
predicciones_svm = svm.predict(X_test)
```

Ejemplo común de aprendizaje supervisado: Clasificación binaria



Recordemos cómo funcionan los modelos de Ciencia de Datos

Entrenamiento
(creación de modelo)

Prueba
(validación)

Producción
(uso de modelo)

Diagram illustrating the structure of a dataset table:

- Columns:** Name, Team, Number, Position, Age
- Rows:** 0 to 6
- Data:** The table contains 7 rows of data for Boston Celtics players.

	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

```
from sklearn.ensemble import RandomForestClassifier
classifier = RandomForestClassifier(n_estimators=100, random_state=1234)
classifier.fit(X_train, y_train)
y_score = classifier.predict_proba(X_test)
```

A Random forest algorithm

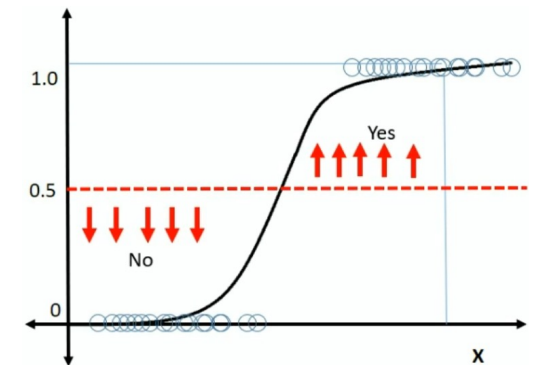
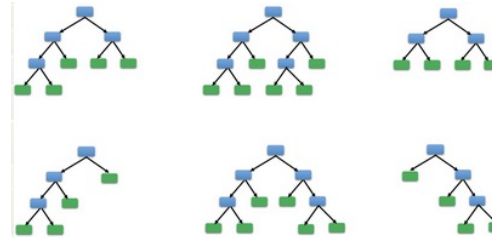
Input: GWAS dataset with M SNPs to analyze
t: number of trees to build

Output: An ensemble of trees

For i=1 to t:

- Select a bootstrap sample from the GWAS dataset
- Grow a tree from the bootstrapped data, by repeating the following steps for each terminal node of the tree:
 1. Select a random subset of variables from the bootstrap sample
 2. Find the variable that best splits the data into homogeneous groups
 3. Split the terminal node into two child nodes

End for



Creación

datos

Columns					
	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Rows

Data

EG

A Random forest algorithm

Input: GWAS dataset with M SNPs to analyze
t: number of trees to build

Output: An ensemble of trees

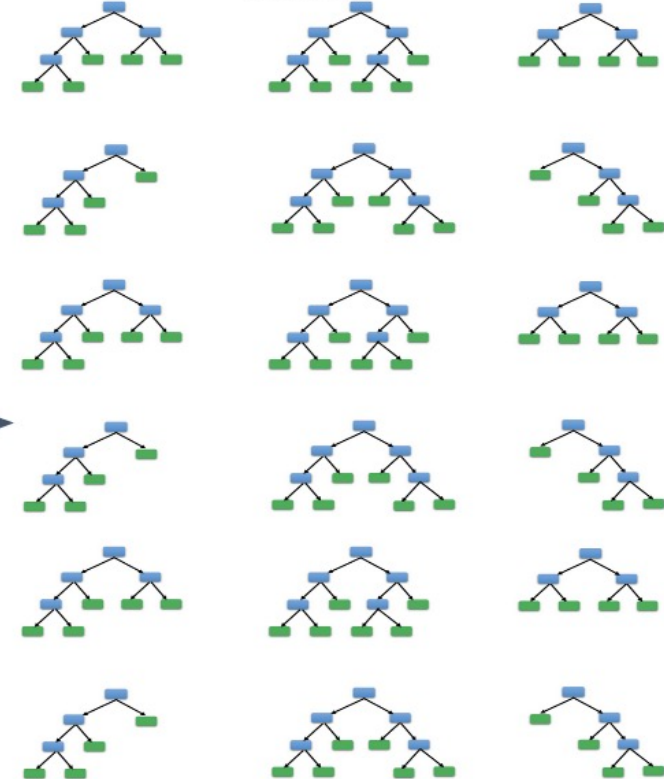
For i=1 to t:

- Select a bootstrap sample from the GWAS dataset
- Grow a tree from the bootstrapped data, by repeating the following steps for each terminal node of the tree:
 1. Select a random subset of variables from the bootstrap sample
 2. Find the variable that best splits the data into homogeneous groups
 3. Split the terminal node into two child nodes

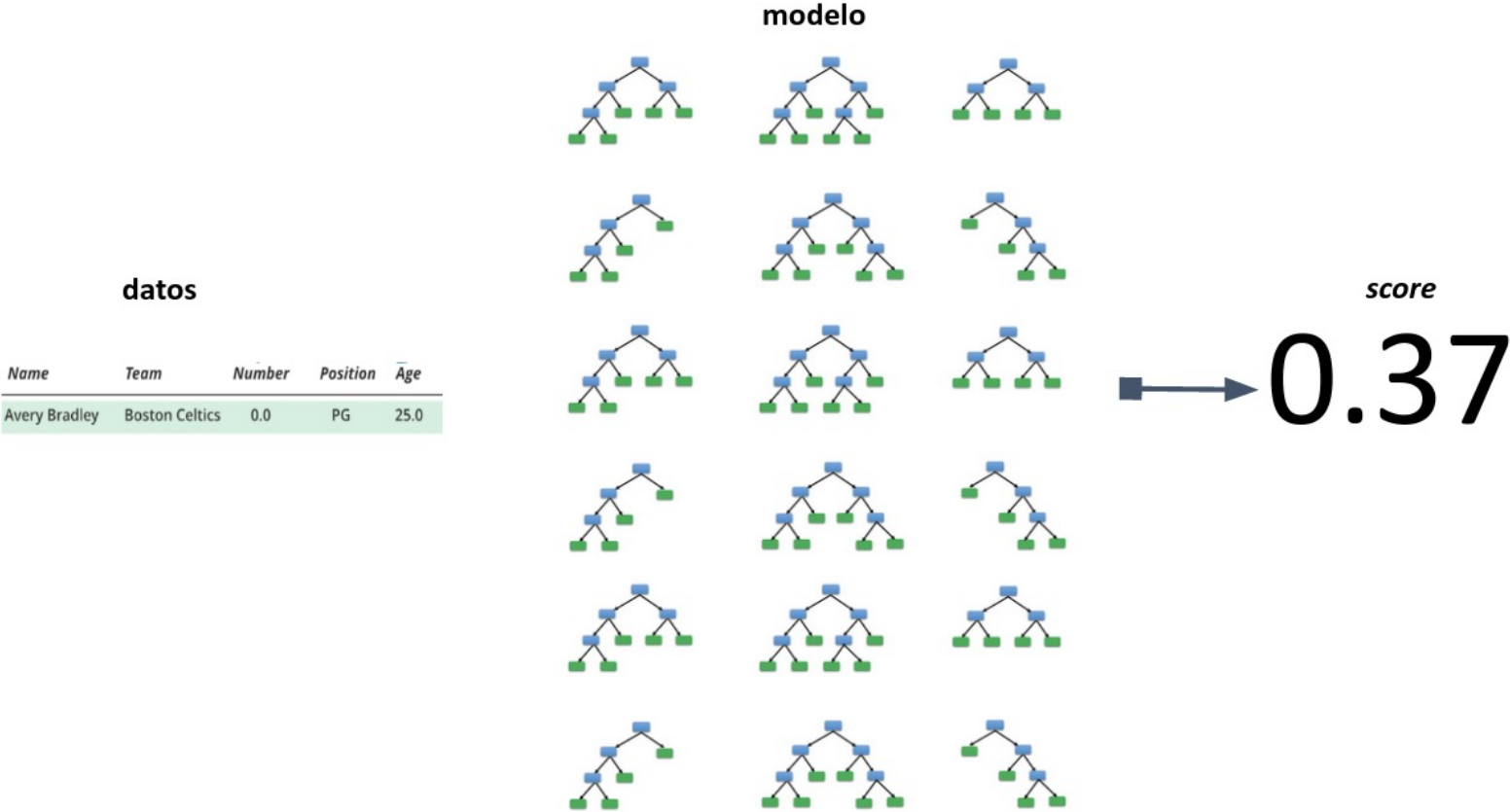
End for

algoritmo

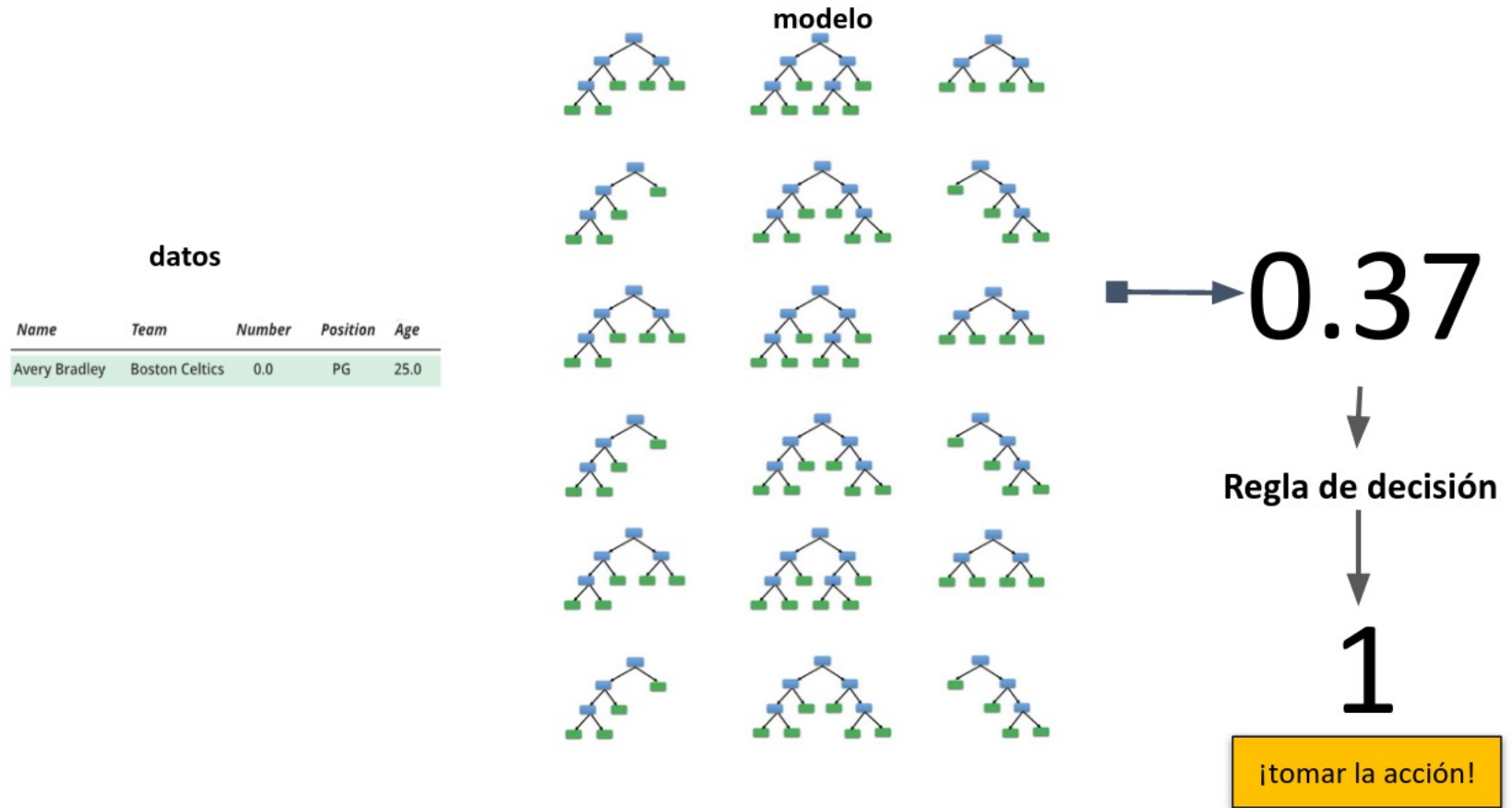
modelo



Uso...



... en los modelos de clasificación, para poder tomar una decisión, necesitamos un “corte” (*threshold*)



Por ejemplo: una transacción que tiene 0.01 de probabilidad de ser fraudulenta, ¿hay que bloquearla? ¿Qué tal una que tiene 0.95? ¿Qué tal una que tiene 0.25?